Parallel Computing Assignment

Group Members -

Ayush Jain	2022B3A70551P
Arpita Tomar	2022B4A70596P

1. Task A -

The objective is to find the top 10 highest-rated movies by averaging all individual user ratings. To process the data efficiently on the GPU, string based movie IDs are first mapped to unique integers on the CPU and passed to the device as flat, parallel 1D arrays. For this, a file is created for extracting the movie id and rating from each review named as ratings_extracted.py. The kernel employs a strict mapping where each CUDA thread processes exactly one review record by using its global thread ID as a direct array index, avoiding internal loops. To aggregate the ratings per movie across all reviews without race conditions, the threads utilize `atomicAdd` on global memory to safely accumulate rating sums and review counts. Once the global reduction is complete, the data is copied back to the host, where the CPU calculates the final averages and sorts the top 10 results. We are calculating two types of time, one is memory copy from host to device and the other is raw kernel time taken for computations. This is done because in part b we will be doing optimizations on the mem copy as well. The result is following-

```
root@f2ad35a36d19:/workspace/assignment# ./cuda_toprated
Loading and parsing data on CPU...
Total Reviews: 1697533
Total Unique Movies: 50052
Launching Kernel...

============================================
Top 10 Rated Movies
============================================
1. ASIN: 6301115538 | Avg Rating: 5 | Reviews: 33
2. ASIN: 0985265108 | Avg Rating: 5 | Reviews: 30
3. ASIN: B008N8A3DA | Avg Rating: 5 | Reviews: 30
4. ASIN: B000OVLBH6 | Avg Rating: 5 | Reviews: 28
5. ASIN: B007ZR9H5K | Avg Rating: 5 | Reviews: 25
6. ASIN: B008F283XG | Avg Rating: 5 | Reviews: 24
7. ASIN: B00B58FWGI | Avg Rating: 5 | Reviews: 24
8. ASIN: B00AQJANI4 | Avg Rating: 5 | Reviews: 23
9. ASIN: B00CYLRQ46 | Avg Rating: 5 | Reviews: 23
10. ASIN: B000H0MNIU | Avg Rating: 5 | Reviews: 22

============================================
             PERFORMANCE METRICS
============================================
Host-to-Device (H2D) Time : 3.22323 ms
Kernel Execution Time     : 0.317312 ms
--------------------------------------------
Total Time (H2D + Kernel) : 3.54054 ms
============================================
```

2. Task B -

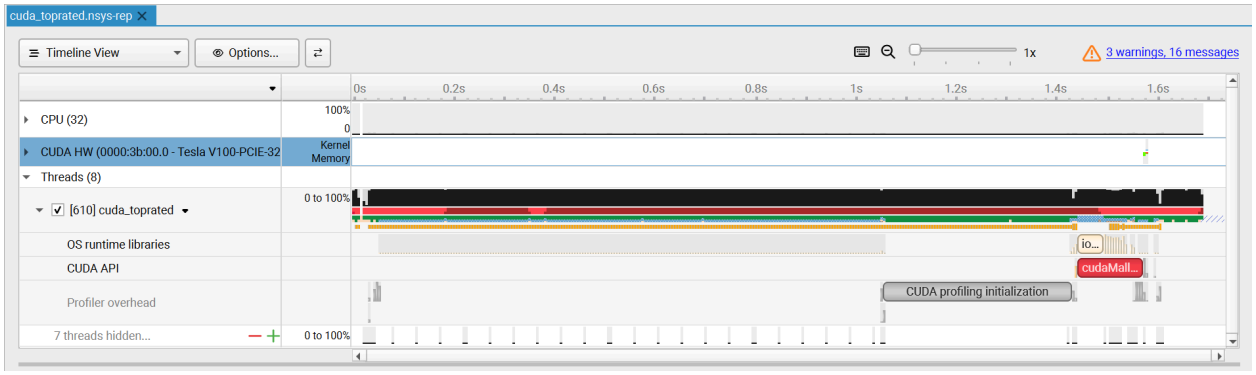
For the optimized CUDA implementation (Task B), the architecture was significantly redesigned to maximize hardware utilization and minimize memory bottlenecks. To eliminate thread divergence, the input arrays were padded with a dummy ID to a perfect multiple of the block size, ensuring all warps execute identical instructions without requiring conditional boundary checks. Memory accesses were optimized by loading coalesced and perfectly aligned global data into shared memory arrays before processing. To drastically reduce global memory contention, the kernel employs warp-level primitives (`__shfl_sync`) to perform an intra-warp reduction, threads dynamically identify peers processing the same movie within the warp, aggregate their ratings directly inside local registers, and elect a single "leader" thread to perform just one `atomicAdd` to global memory. Finally, to hide PCIe transfer latency, host arrays were allocated using pinned memory (`cudaMallocHost`), allowing `cudaMemcpyAsync` to divide the dataset across four concurrent CUDA streams, effectively overlapping data transfers with kernel computation. The result is as follows -

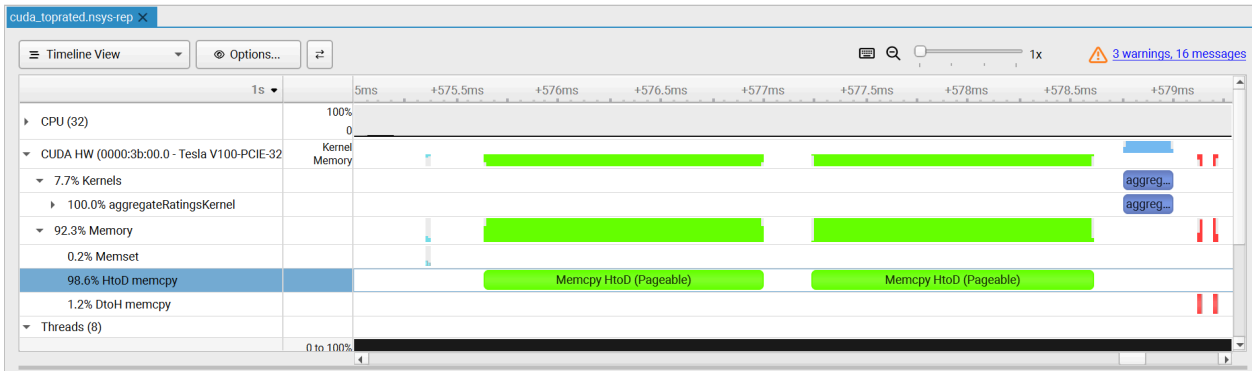
```
root@f2ad35a36d19:/workspace/assignment# ./cuda_toprated_optimized
Loading data from CSV...
Launching Optimized Kernel with Streams...

==========================================
Top 10 Rated Movies (Optimized)
==========================================
1. ASIN: 6301115538 | Avg Rating: 5 | Reviews: 33
2. ASIN: 0985265108 | Avg Rating: 5 | Reviews: 30
3. ASIN: B008N8A3DA | Avg Rating: 5 | Reviews: 30
4. ASIN: B000OVLBH6 | Avg Rating: 5 | Reviews: 28
5. ASIN: B007ZR9H5K | Avg Rating: 5 | Reviews: 25
6. ASIN: B008F283XG | Avg Rating: 5 | Reviews: 24
7. ASIN: B00B58FWGI | Avg Rating: 5 | Reviews: 24
8. ASIN: B00AQJANI4 | Avg Rating: 5 | Reviews: 23
9. ASIN: B00CYLRQ46 | Avg Rating: 5 | Reviews: 23
10. ASIN: B000H0MNIU | Avg Rating: 5 | Reviews: 22

==========================================
             PERFORMANCE METRICS
==========================================
Stream 0 Kernel Time    : 0.0728 ms
Stream 1 Kernel Time    : 0.038944 ms
Stream 2 Kernel Time    : 0.036768 ms
Stream 3 Kernel Time    : 0.019488 ms
------------------------------------------
Sum of Stream Kernels     : 0.168 ms
Total Time (H2D + Kernel) : 1.27078 ms
==========================================
```

3. Task C -

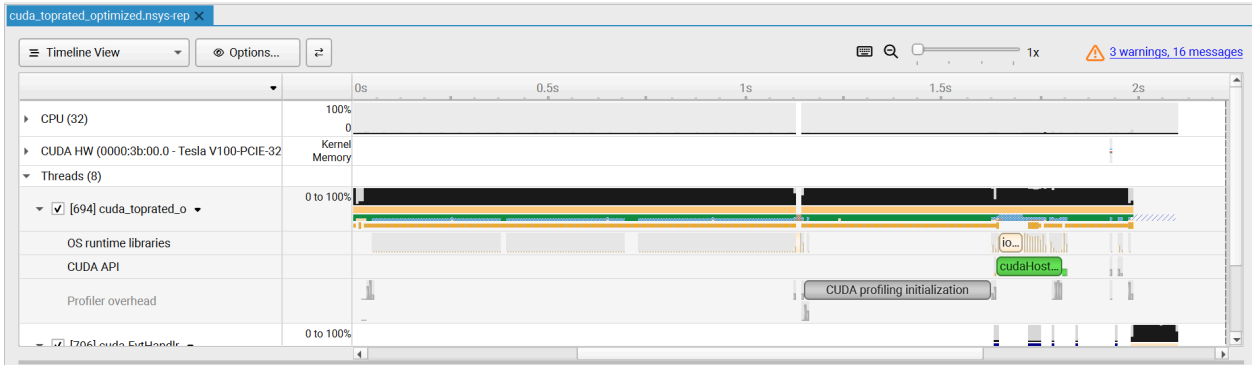
The time taken in part A for computation as well as mem copy instruction is significantly higher than part B. The raw computation speedup is $\frac{0.317312}{0.168} = 1.89$ whereas the overall speedup including memory operations and computation is $\frac{3.54054}{1.27078} = 2.79$. This clearly suggests that memory copying is

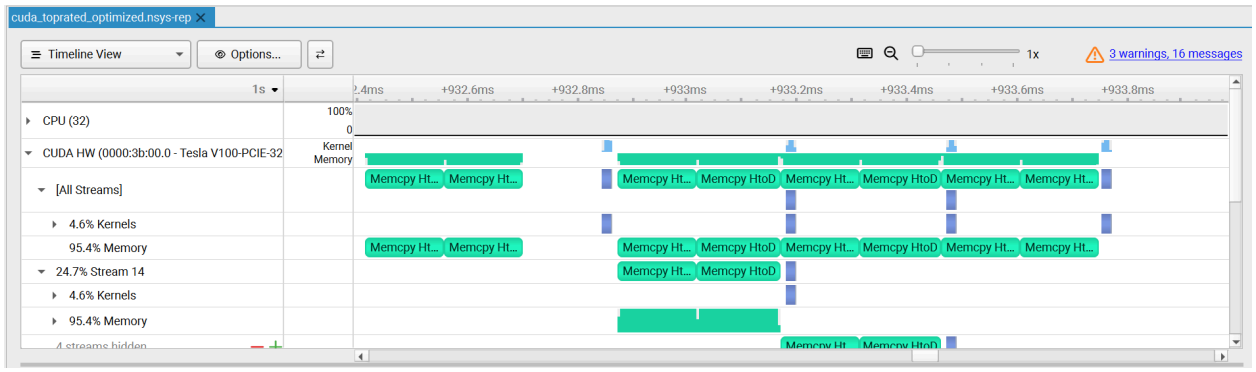
the major bottleneck task while computations are getting done in relatively very less amount of time. The memory copying bottleneck is reduced in part B by using streams where the data gets copied in the gpu ram while gpu is computing one set of results, thereby doing things concurrently. The following is the Nsight Systems report for part A -





And the following is the same report for part B -





Now, following is the Nsight Compute report for part A -

Compute (SM) Throughput [%]	12.80
Memory Throughput [%]	20.56
L1/TEX Cache Throughput [%]	31.32
L2 Cache Throughput [%]	20.56
DRAM Throughput [%]	7.23

Memory Throughput [Gbyte/s]	62.83
L1/TEX Hit Rate [%]	0
L2 Hit Rate [%]	86.79

Lg Throttle Stalls **Est. Speedup: 79.44%**	On average, each warp of this kernel spends 997.8 cycles being stalled waiting for the L1 instruction queue for local and global (LG) memory operations to be not full. Typically, this stall occurs only when executing local or global memory instructions extremely frequently. Avoid redundant global memory accesses. Try to avoid using thread-local memory by checking if dynamically indexed arrays are declared in local scope, of if the kernel has excessive register pressure causing by spills. If applicable, consider combining multiple lower-width memory operations into fewer wider memory operations and try interleaving memory operations and math instructions. This stall type represents about 81.6% of the total average of 1223.5 cycles between issuing two instructions.

Following is the similar report for part B -

Compute (SM) Throughput [%]	70.60
Memory Throughput [%]	72.00
L1/TEX Cache Throughput [%]	81.58
L2 Cache Throughput [%]	12.59
DRAM Throughput [%]	26.31

Memory Throughput [Gbyte/s]	226.53
L1/TEX Hit Rate [%]	0
L2 Hit Rate [%]	20.47

FP32 Non-Fused Instructions **Est. Speedup: 19.46%**	This kernel executes 0 fused and 424448 non-fused FP32 instructions. By converting pairs of non-fused instructions to their fused, higher-throughput equivalent, the achieved FP32 performance could be increased by up to 50% (relative to its current performance). Check the Source page to identify where this kernel executes FP32 instructions. ▶ Key Performance Indicators

The throughput has clearly increased in part B among all dimensions.

a. **Biggest Overall Pipeline Gain: CUDA Streams** The profiling data clearly indicates that Host-to-Device (H2D) memory transfer was the primary bottleneck of the application. In the baseline implementation (Task A), the memory copy took roughly 3.22 ms while the raw computation only took 0.31 ms. By utilizing pinned memory (cudaMallocHost) and deploying four concurrent CUDA streams, the optimized version (Task B) successfully overlapped the PCIe data transfers with active kernel computation. This latency-hiding technique was the primary driver for reducing the total pipeline time from 3.54 ms to 1.27 ms, achieving an impressive 2.79x overall speedup.

b. **Biggest Hardware Efficiency Gain: Warp-Level Reduction** For the raw GPU computation, replacing global atomicAdd calls with intra-warp reduction using __shfl_sync contributed the most to hardware efficiency. The Nsight Compute report for Task A highlights massive "Lg Throttle Stalls," indicating that warps were heavily stalled waiting for global memory queues to resolve atomic collisions. By aggregating the ratings directly inside local thread registers and electing a warp leader to perform a single global write, the memory contention was drastically reduced. As a result, the Compute (SM) Throughput skyrocketed from 12.80% to 70.60%, and the active Memory Throughput nearly quadrupled from 62.83 GB/s to 226.53 GB/s.

4. Task d -

The program analyzes the review text and labels it as positive, negative or neutral based on sentiment score.

Starting from `main()`, the program first loads the VADER lexicon on the CPU, tokenizes each lexicon phrase into integer word IDs, flattens those phrases into compact arrays, and copies the lexicon data into CUDA constant memory, while also building a candidate index by first token for faster matching. It then opens the Amazon review dataset, parses each JSON line with `cJSON`, extracts `reviewText`, tokenizes the review into the same word IDs, and groups reviews into batches. For each batch, the tokenized reviews are copied to the GPU, where one CUDA thread processes one review by scanning its token sequence, checking matching lexicon phrases, accumulating the sentiment score, and assigning a positive, negative, or neutral label. After all batches are processed, the program writes the labels to an output file and prints the final counts along with kernel time and total program time.

```
root@f2ad35a36d19:/workspace/assignment# ./cuda_reviewanalysis_2
Loaded lexicon entries: 7380
Maximum phrase length: 4

Summary
Total JSON lines read: 1697533
Valid reviews processed: 1697533
JSON parse errors: 0
Missing reviewText: 0
Positive: 1436922
Negative: 244921
Neutral: 15690
Total kernel time: 79.9104 ms
Total program time: 37.6512 s
Labels written to: review_sentiment_labels.txt
```

5. Task e -

For storing reviewerID a structure storing reviewerID, count of elaborate reviews, total words in elaborate reviews and an int used is made. A hashtable from these structures is made for hashmap. The code parses the JSON file using cJSON, it concurrently reads and identifies if the the reviewText for a given reviewerID is longer than 50 words, if so the reviewerID is added into the hashmap created using standard djb2 hash function or if the reviewerID is already present the count is incremented by one and the total_words is incremented by words in reviewText. All the Reviewer IDs along with elaborate review count and elaborate review average length is printed on screen. Finally a statistics of total reviews, elaborate reviews is printed along with total execution time (including file reading+ processing).

```
ReviewerID: A3RGMYN9CBTLP7, Count: 6, Average Review Length: 69.83 words
ReviewerID: A3N110LGDXE6FH, Count: 11, Average Review Length: 488.45 words
ReviewerID: A33YN5F6MF1R8R, Count: 8, Average Review Length: 59.25 words
ReviewerID: A330WTOST03ENT, Count: 24, Average Review Length: 72.12 words
ReviewerID: A3FHV3RV8Z12E6, Count: 5, Average Review Length: 98.60 words
ReviewerID: A2H03MA51T7YXC, Count: 5, Average Review Length: 71.60 words
ReviewerID: A3C83E6ZYSZUMM, Count: 23, Average Review Length: 75.17 words
ReviewerID: A2BNL3575OCADM, Count: 5, Average Review Length: 94.20 words

Statistics
Total reviews processed: 1697533
Elaborate reviews: 1112715
Parse errors: 0
Execution time: 39.966394 seconds
```

Measuring time without printing output

```
> time ./c_elaborate

Statistics
Total reviews processed: 1697533
Elaborate reviews: 1112715
Parse errors: 0
Execution time: 31.309734 seconds

real    0m31.329s
user    0m30.064s
sys     0m1.232s
```

6. Task f -

The program uses a similar hashmap idea as the previous question along with an additional partialEntry data structure to store intermediate parallel results.

The program starts by opening the JSON reviews file, reading every line into memory, and starting a timer. It then runs an OpenMP parallel loop where each thread parses one review line with cJSON, extracts `reviewerID` and `reviewText`, counts the words, and keeps only “elaborate” reviews with at least 50 words in a per-line intermediate result array while also tracking totals and parse errors through reductions. After the parallel phase, it sequentially merges the valid results into a hash table keyed by reviewer ID, computes each reviewer’s count and average review length, prints only reviewers with at least 5 elaborate reviews, and finally prints overall statistics and execution time before freeing memory.

```
ReviewerID: A3RGMYN9CBTLP7, Count: 6, Average Review Length: 69.83 words
ReviewerID: A3N110LGDXE6FH, Count: 11, Average Review Length: 488.45 words
ReviewerID: A33YN5F6MF1R8R, Count: 8, Average Review Length: 59.25 words
ReviewerID: A330WTOST03ENT, Count: 24, Average Review Length: 72.12 words
ReviewerID: A3FHV3RV8Z12E6, Count: 5, Average Review Length: 98.60 words
ReviewerID: A2H03MA51T7YXC, Count: 5, Average Review Length: 71.60 words
ReviewerID: A3C83E6ZYSZUMM, Count: 23, Average Review Length: 75.17 words
ReviewerID: A2BNL3575OCADM, Count: 5, Average Review Length: 94.20 words

Statistics
Total reviews processed: 1697533
Elaborate reviews: 1112715
Parse errors: 0
Execution time: 6.158468 seconds
```

The task execution on different threads

OMP_NUM_THREADS=2

```
> OMP_NUM_THREADS=2 time ./c_elaborate_openmp

Statistics
Total reviews processed: 1697533
Elaborate reviews: 1112715
Parse errors: 0
Execution time: 23.358027 seconds
33.26user 6.40system 0:25.01elapsed 158%CPU (0avgtext+0avgdata 2339856maxresident)k
3877792inputs+0outputs (0major+1570146minor)pagefaults 0swaps
```

OMP_NUM_THREADS=4

```
> OMP_NUM_THREADS=4 time ./c_elaborate_openmp

Statistics
Total reviews processed: 1697533
Elaborate reviews: 1112715
Parse errors: 0
Execution time: 13.534895 seconds
35.65user 4.24system 0:14.77elapsed 270%CPU (0avgtext+0avgdata 2340056maxresident)k
0inputs+0outputs (0major+1455345minor)pagefaults 0swaps
```

OMP_NUM_THREADS=8

```
> OMP_NUM_THREADS=8 time ./c_elaborate_openmp

Statistics
Total reviews processed: 1697533
Elaborate reviews: 1112715
Parse errors: 0
Execution time: 8.880732 seconds
38.72user 5.42system 0:11.41elapsed 386%CPU (0avgtext+0avgdata 2340232maxresident)k
0inputs+0outputs (0major+1182715minor)pagefaults 0swaps
```

OMP_NUM_THREADS=16

```
> OMP_NUM_THREADS=16 time ./c_elaborate_openmp

Statistics
Total reviews processed: 1697533
Elaborate reviews: 1112715
Parse errors: 0
Execution time: 6.823076 seconds
45.07user 4.27system 0:07.85elapsed 628%CPU (0avgtext+0avgdata 2339528maxresident)k
0inputs+0outputs (0major+890167minor)pagefaults 0swaps
```

7. Task g -

Amdahl's law says:

$$S(N) = \frac{1}{(1-P)+\frac{P}{N}}$$

where:

- S(N) = speedup with N processors/threads
- P = parallelizable fraction of the program
- 1−P = serial fraction

Speedup= $\frac{parallel\ time}{serial\ time}$

For the full dataset, the sequential `c_elaborate` took **31.309734 s**, while `c_elaborate_openmp` took **23.358027 s (2 threads)**, **13.534895 s (4 threads)**, **8.880732 s (8 threads)**, and **6.823076 s (16 threads)**. This shows that the OpenMP version consistently outperforms the sequential version, and the best measured runtime is obtained at 16 threads.
Using the sequential runtime as the baseline, the speedups are:

- **2 threads:** 31.309734/23.358027=1.34×

- **4 threads:** 31.309734/13.534895=2.31×
- **8 threads:** 31.309734/8.880732=3.53×
- **16 threads:** 31.309734/6.823076=4.59×

The scaling is good, but clearly **sub-linear**. Parallel efficiency falls as thread count increases:

- **2 threads:** 1.34/2=67.0%
- **4 threads:** 2.31/4=57.8%
- **8 threads:** 3.53/8=44.1%
- **16 threads:** 4.59/16=28.7%

This behavior is exactly what Amdahl's Law predicts: even if a large part of the program is parallel, the remaining serial part limits total speedup. From the observed speedups, the implied serial fraction drops toward roughly **16–18%** at higher thread counts, which means the theoretical maximum speedup is only about **5.5× to 6.0×**, even with very many threads. That is why doubling threads from **8 to 16** improves runtime only from **8.88 s to 6.82 s**, not by another 2×.

In this program, the Amdahl's Law limitation is likely caused by the parts that are not fully parallelized, especially the work that still happens sequentially after the parallel review-processing phase, along with file I/O, memory allocation, JSON parsing overhead, and OpenMP thread-management / memory-bandwidth costs. So, the OpenMP version gives a strong improvement over the sequential version, but the results also show diminishing returns as thread count increases because the serial and overhead portions begin to dominate.